

Lab0: GitLab 实验报告

一、问题回答

1. 多人协同开发经历

有。OOP 大作业中，用 GitHub 建仓库，便于查看和修改。平时和同学分享资料也用 GitHub，分支明确。

2. 为什么要“暂存-提交”两步

- **精细控制提交内容：**工作区里常有几处不相关的修改，暂存区可以只挑相关部分提交，让每个 commit 只做一件事。
- **提交前可以检查：**add 后可用 `git diff --staged` 查看，有问题可撤回，不会污染提交历史。

3. `git branch` 与 `git branch -a` 的区别

`git branch` 只列出本地分支；`git branch -a` 同时列出远程跟踪分支（以 `remotes/origin/` 开头）。

二、实验步骤

1. 环境准备

安装 Git 并配置用户名、邮箱；生成 SSH 密钥添加到 GitHub；用模板创建个人仓库，在 VSCode 中克隆到本地。

2. 完成 TODO（任务 2）

在 `main.c` 的 TODO 处添加 `printf("Hello, ics!\n");`，提交为 `complete TODO in main.c` 并推送。

3. 合并冲突（任务 4）

创建 `feature` 分支，在 `main.c` 第 7 行添加 `printf("feature: hello world\n");` 并提交。

切回 `main`，在同一行添加 `printf("main:hello\n");` 并提交。两个分支由此分叉：

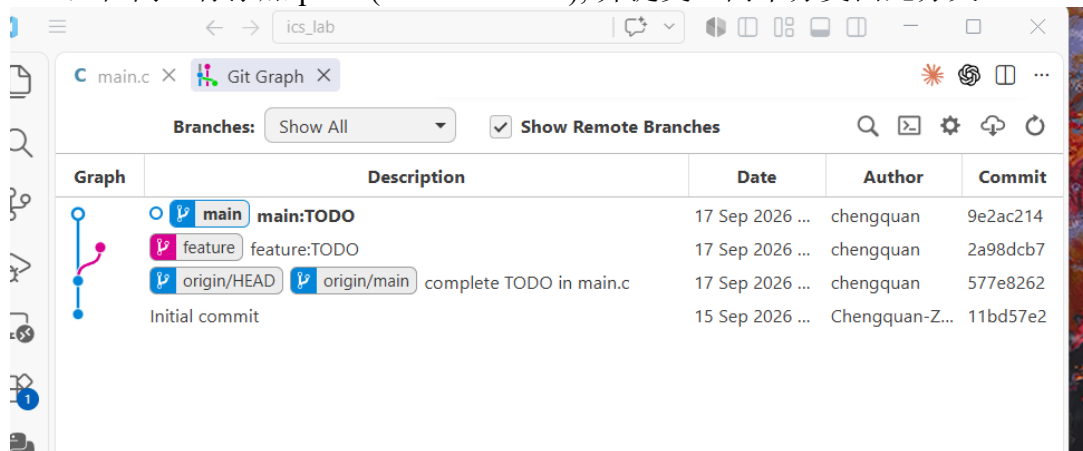


图 1 合并前的分支图

在 `main` 上合并 `feature`。两个分支修改了同一行，产生冲突：

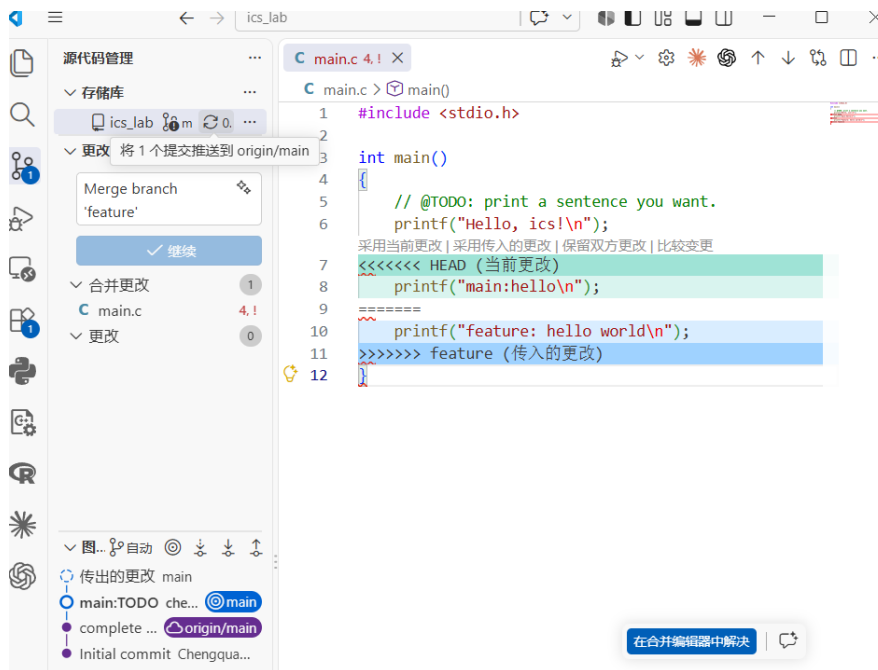


图2 合并冲突

选择“采用当前更改”，保留 main 的内容：

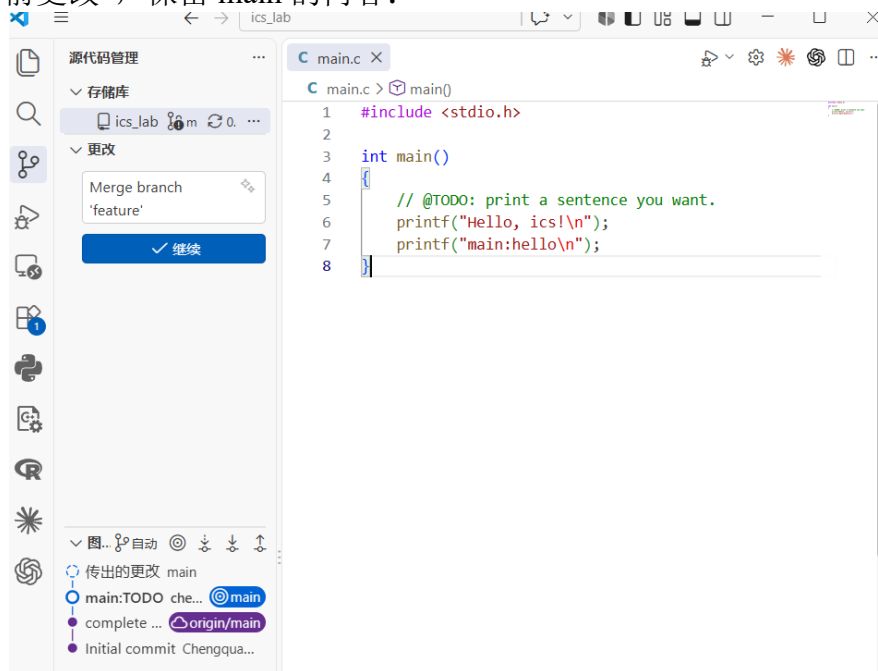


图3 冲突解决后的 main.c

暂存并提交 Merge branch 'feature', 推送到远程。feature 汇入 main，形成合并提交：

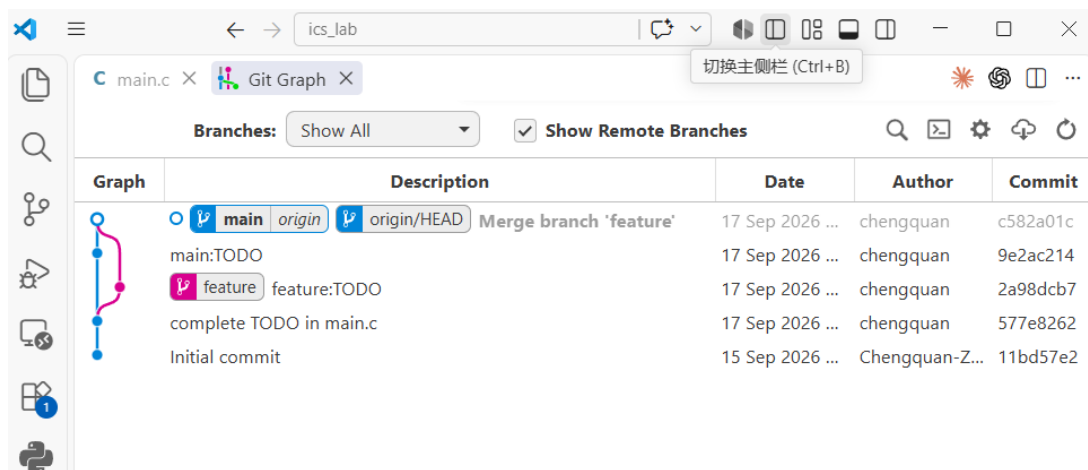


图 4 合并后的分支图

三、阅读总结（任务 3）

1. Commit Message 规范

介绍 Angular 规范：提交信息分 Header、Body、Footer 三部分。Header 必填，格式为 type(scope): subject, type 如 feat、fix、docs、refactor 等。规范的提交信息便于浏览、筛选历史，并能自动生成 Change Log。

2. Git Flow 分支控制

介绍 Gitflow 模型：master 存放线上版本，develop 是公共开发分支，feature 用于开发新功能，release 用于测试和发布，hotfix 用于紧急修复线上 bug。

3. 为什么学 Git

- **可回溯：**每次提交都是快照，改坏了随时能回退。
- **便于协作：**分支让多人并行开发，冲突也会被明确标出。
- **规范提效：**提交规范和分支策略让团队协作更有序。